Package

1. Pengertian ***package.***
2. Pengaruh ***package*** terhadap method main().
3. Membuat ***package.***
4. Mendeklarasikan dan memberi nama ***package.***
5. Membuat struktur direktori.
6. ***Compile*** dan ***run*** suatu ***class*** dalam suatu ***package.***
7. Menggunakan ***package***.

Pengertian *package*

- Package adalah sarana **taksonomi** dan **pengorganisasian** Class-class dan Interface yang **sekelompok menjadi suatu unit** tunggal dalam library (seluruh class yg ada di dalamnya sudah dikompilasi semua).
- Representasi package **berupa folder** yang berisi file-file class/interface/enum/package lain yang kegunaannya hampir sama sehingga patut untuk dikelompokkan. **Kecuali** class yang mengandung **method main()** tidak perlu dikelompokkan.
- Package mempengaruhi **mekanisme hak akses** ke Class-class di dalamnya (Kerjasama antar class dipengaruhi keberadaan package).

Pengaruh *package* terhadap method main()

- Class utama yang mengandung method main() sebaiknya tidak berada dalam suatu package non standard.
- Class utama yang mengandung method main() boleh berada dalam **package standard** yang bernama **default**, sehingga hirarki posisi folder tempat class utama yang mengandung method main() **di atas/sejajar** dengan package yang diimport.

Membuat *package*

Ada tiga langkah untuk membuat package:

1. **Mendeklarasikan** dan memberi **nama** package.
2. **Membuat struktur** dan **nama folder** yang sesuai dengan struktur dan nama package.
3. **Mengkompilasi** seluruh Class-class di dalam packagenya masing-masing.

Mendeklarasikan dan memberi nama *package*

- Deklarasi package wajib diletakkan pada bagian **paling awal** (sebelum deklarasi import) dari ***source code*** setiap ***class*** yang dibungkus ***package*** tersebut.
- Template/bentuk umum deklarasi package:
- ***package namaPackage;***
- Deklarasi tersebut akan memberitahukan interpreter library class atau interface yang manakah yang dirujuk dan digunakan.

Syarat nama package

Syarat nama package sama dengan nama identifier pada umumnya, dengan syarat tambahan:

1. Diawali huruf kecil.
2. Menggambarkan class-class yang dibungkusnya.
3. Harus unik (berbeda dengan nama package lainnya termasuk package standard).
4. Merepresentasikan path dan package tsb.
5. Harus sama dengan nama folder/direktory-nya.

Contoh package standard

- Berisi class-class fundamental yang sering digunakan: ***java.lang;***
- Berisi class-class yang membangun aplikasi GUI: ***java.awt; javax.swing;***
- Berisi class-class untuk proses input/output: ***java.io;***

Membuat struktur direktori/folder

- Pada langkah ini, buatlah folder menggunakan file manager (di windows menggunakan explorer) sesuai struktur package dari langkah sebelumnya.
- Tempatkan class-class tersebut ke dalam folder yang bersesuaian (simpan file-file ke dalam foldernya).
- Package dapat bersarang di package lain, sehingga dapat dibuat struktur/hirarki package.
- Template/bentuk umum deklarasi package multilevel:
- *package namapackage1[.namapackage2[.namapackage3]];*
- Contoh hirarki package standard di JDK:
- ***package java.awt.image;***

Compile dan *run* suatu *class* dalam suatu *package*

- Langkah selanjutnya, setiap class dalam package yang telah dibuat, masing-masing ***dikompilasi*** menjadi byte code (*.class), sehingga package tersebut siap digunakan.

Menggunakan *package*

Ada dua cara menggunakan suatu package:

1. Class yang menggunakan suatu package, berada dalam direktory/folder (package) yang sama dengan class-class lain yang digunakan/dipanggil, sehingga ***tidak*** diperlukan ***import***.

2. Class yang menggunakan suatu package, berada dalam direktory /folder (package) yang berbeda dengan class- class yang digunakan, sehingga pada awal *source code* di class yang menggunakan, jadi template-nya harus mencantumkan:

- ***import namaPackage.Namaclass;*** atau
- ***import namaPackage.*;***

- Contoh:
- ***import java.text.DecimalFormat;***
- ***import javax.swing.*;***